

弱标签、引擎评分与调参流程对话整理

基于 test1 项目 engine.build_dataset / engine_store / evaluate_params / pipeline 代码逻辑

1. 数据集生成命令与用途

```
.\.venv\Scripts\python.exe -m engine.build_dataset --conflict-only
```

该命令从 engine_detections 表中选择两个引擎都有记录的 MD5，合并 360/cm 字段，生成 weak_label，并按 MD5 稳定切分 train/val/test。

项目	说明
输出目录	data\datasets
输出文件	train.jsonl、val.jsonl、test.jsonl
train	用于分析规律，后续可作为训练候选数据。当前项目尚未真正用 train 训练模型。
val	用于调融合参数，必须使用 val 中的 weak_label 计算指标。
test	用于最终验收，不应反复拿来调参。

2. 360/cm 字段如何合并

合并逻辑位于 engine_store.py 的 build_sample_from_engine_records()。当前 MVP 中，360 被视为 Engine A，cm 被视为 Engine B。

```
360 score -> engine_a_score
cm score  -> engine_b_score
```

- 文本字段如 app_name、package_name、app_type、platform、control_url、download_url、virus_name、fraud_family、sdk_list，会按记录顺序取第一个非空值。
- fake_app 或 impersonation_flag 任一为真，则合并样本的 fake_app 为 True。
- steady 字段显示不是未加固/未知/空时，packer 置 True。
- 存在 cert_md5 / cert_sha1 / cert_sha256 时，signature_status 置为 normal。
- 原始 360/cm 记录会保留在 engine_records 中，便于追溯。

3. score 从哪里来，作用是什么

engine_detections.score 来自导入的 360.xlsx / cm.xlsx 的 score 列，项目只读取和使用该分数，不在代码里反推 360/cm 内部评分算法。

项目	说明
来源	Excel 的 score 列，导入到 data\mvp.db 的 engine_detections.score。
解释方式	当前项目把它视为 0-100 的原始引擎风险强度分。
单引擎标签	score >= 70 为 malicious; score >= 30 为 suspicious; score < 30 为 benign。
后续用途	生成 engine_a_score / engine_b_score、weak_label、冲突识别和融合

4. weak_label 在哪里定义，怎么定义

弱标签定义在 build_dataset.py 的 weak_label(sample) 函数中。它是规则标签，不是人工金标准。

```
if fraud_family 非空或存在仿冒分类字段:
    weak_label = "malicious"
elif min(engine_a_score, engine_b_score) >= 70:
    weak_label = "malicious"
elif max(engine_a_score, engine_b_score) < 30:
    weak_label = "benign"
elif abs(engine_a_score - engine_b_score) >= 35:
    weak_label = "suspicious"
elif max(engine_a_score, engine_b_score) >= 45:
    weak_label = "suspicious"
else:
    weak_label = "benign"
```

因此，360=0、cm=93 这类强冲突样本通常会被弱标签标为 suspicious；两个引擎都高于 70 才会因分数直接标为 malicious。

5. fraud_family 是什么

fraud_family 是样本的涉诈/恶意家族字段，来自导入的引擎或业务数据。当前规则把它当成强风险信号。

```
"virus_name": "Trojan.Fraud.Sex"
"fraud_family": "J-色情视频-46b48d783"
```

只要 fraud_family 非空, weak_label 会优先标为 malicious。这是启发式规则, 并不等同于人工确认。

6. Engine C: 真实流程与近似调参分的区别

真实研判流程里的 Engine C 并不是 evaluate_params.py 里的简单加分。真实 Engine C 位于 pipeline.py, 流程是多智能体证据块、双模型辩论和仲裁。

```
sample
-> extract_iocs()
-> run_agents()
-> static_analysis_agent / threat_intel_agent / impersonation_agent /
business_label_agent
-> debate()
-> model_a_score / model_b_score
-> arbiter.score
-> wec_decision()
```

evaluate_params.py 里的 engine_c_score 是 Fast Engine-C-like score, 只是为了避免在 1 万多条 val 样本上逐条调用本地 Qwen, 因此用结构化规则近似 Engine C。

结论: 近似版可以用于快速粗调参数范围, 但不能严格代表真实 Engine C 的准确率。真实验收应使用真实 Engine C 和人工标签。

7. final_score 和阈值是什么意思

final_score 是把 360 分、cm 分和 Engine C 分融合后的最终风险分, 范围通常为 0 到 1。

```
final_score >= malicious_threshold -> malicious
final_score >= suspicious_threshold -> suspicious
otherwise -> benign
```

例如 malicious_threshold=0.80、suspicious_threshold=0.50 时, final_score >= 0.80 判恶意, 0.50 到 0.80 判可疑, 小于 0.50 判良性。

8. val 调参指标如何排序

evaluate_params.py 当前按以下优先级选参数:

1. risk_recall
2. malicious_recall
3. accuracy

项目	说明
risk_recall	真实 weak_label 为 suspicious 或 malicious 的样本，有多少被预测成 suspicious 或 malicious。重点是不要漏掉风险。
malicious_recall	真实 weak_label 为 malicious 的样本，有多少被预测成 malicious。重点是要漏掉明确恶意。
accuracy	三分类整体一致率。由于弱标签不是金标准，它只能代表和 weak_label 的一致性。

9. 训练、验证、测试的正确理解

当前项目还没有真正用 train.jsonl 训练模型。现有 evaluate_params.py 做的是参数网格搜索，而不是模型训练。

- 如果未来真正训练模型，训练标签可以先使用 train.jsonl 中的 weak_label 或 target.verdict。
- val 阶段必须使用 val 的标签计算 accuracy、recall、precision 和混淆矩阵，否则无法判断参数好坏。
- test 只用于参数固定后的最终验收，不应用来反复调参。
- 由于 weak_label 不是人工金标准，后续最好从 review_candidates.csv 中人工复核，形成 gold_label。

10. review_candidates.csv 在什么情况下出现样本

review_candidates.csv 是 evaluate_params.py 自动挑出的建议人工复核样本。它不是随机样本，而是优先暴露规则问题、弱标签问题和引擎冲突问题的样本。

项目	说明
mismatch	当前参数预测 pred 与 val.jsonl 中 weak_label 不一致。注意这里不是和真实标签不一致。
engine_conflict	360/cm 分差 ≥ 35 ，例如 360=0、cm=93。
high_risk	final_score $\geq$ malicious_threshold，或 fake_app 为真，或 fraud_flag 为空。
boundary	final_score 靠近 suspicious/malicious 阈值，稍微调参就可能改变分类。

mismatch 的准确含义是 `weak_label != pred`。当前 `val.jsonl` 没有人工真实标签，只有 `weak_label`。人工复核之后，才可以补充真正的 `gold_label`。

11. 最重要的结论

- `engine_detections.score` 是 360/cm Excel 中已有的原始风险分，项目不负责计算其内部来源。
- `weak_label` 是 `build_dataset.py` 用引擎分数和业务字段生成的弱标签，不是人工金标准。
- 真实 Engine C 是多智能体 + 双模型辩论 + 仲裁；`evaluate_params.py` 里的 `engine_c_score` 只是为了批量调参的近似分。
- 当前 `val` 上的准确率只能说明与 `weak_label` 的一致性，不能代表真实系统准确率。
- 更严谨的后续流程是：近似版粗调 -> 抽样跑真实 Engine C -> 人工复核形成 `gold_label` -> 再校准阈值和模型。